Verifying Distributed Message Passing Protocols in Rust

How to prove a message-passing protocol by reasoning about one thread at a time.

---

1 The Problem

We consider programs written in a small message-passing language. A program is a fixed set of threads that share no memory and communicate only by passing messages over *channels*. A channel is created dynamically and has two endpoints: a *sender*, which may be held by several threads at once, and a *receiver*, which is held by exactly one. Sending is non-blocking; a thread never waits to hand a message off. Receiving blocks until a message is available.

Which message a receive returns is fixed by the channel's *discipline*. We consider two. Under FIFO, messages are delivered in the order they were sent, and we identify a channel with an ordered pair `(source, destination)`, so a FIFO channel has a single sender. Under BAG, a channel is an unordered pool and a receive returns any message that has not yet been consumed; many threads may send on it.

We assume a Rust library that provides exactly this, and that the library itself has been verified, so that its specification may be taken as given. On top of it we also assume a verified `rpc` primitive, since request/response is the dominant pattern in practice and we would like protocol code to use it directly. Section 9 shows how `rpc` is built and verified from the message-passing operations; Section 16 lists precisely what remains assumed.

The question this note answers is: given such a library, how do we verify a *protocol* written on top of it?

As a running example, consider the two-phase commit protocol written on top of our message-passing library. A coordinator asks each participant whether it can commit; if every answer is yes, it commits.

TWO-PHASE COMMIT IN RUST, ABRIDGED

```
// Coordinator
fn coordinator(peers: &[Peer]) -> bool {
    let mut all_yes = true;
    for p in peers {
        p.req.send(Prepare);              // nonblocking
        match p.rsp.recv() {              // blocks
            Vote(true)  => {}
            _           => all_yes = false,
        }
    }
    for p in peers { p.req.send(Decide(all_yes)); }
    all_yes
}

// Participant
fn participant(rx: Receiver<Msg>, tx: Sender<Msg>, vote: bool) {
    loop {
        match rx.recv() {                 // blocks
            Prepare   => tx.send(Vote(vote)),   // nonblocking
```

```
            Decide(b) => commit_or_abort(b),
            _         => {}
        }
    }
}
```

Both phases of the protocol reach a participant on the same channel, so a single receive serves both: the participant does not know, before it looks, whether the message in front of it is a request to vote or the announcement of the outcome. That is worth noticing because it fixes how many interference points the participant has. It has one per loop iteration — the receive — and not one per protocol phase. The verified version in `src/twophase.rs` has exactly this shape.

That receive does carry a proof obligation — a *yield invariant*, in the sense of Section 8, which the participant may assume when it wakes up and must preserve at each of its own steps. For two-phase commit at this level of abstraction the obligation is discharged trivially, because the property we are after is a property of what is on the reply channels, and the participant only ever appends to them. Section 10 shows a version of the same protocol whose yield invariant is not trivial, and Section 11 one where establishing it is most of the work.

The property we want is:

> If the coordinator commits, then every participant voted yes.

Proving it directly in a verifier like Verus is quite complicated. The coordinator and every participant run concurrently, so the proof must characterize every state reachable under every interleaving of messages. An inductive invariant has to describe, for an arbitrary point in the execution, which requests are in flight, which have been consumed, which replies exist but have not yet been read, and how those sets relate to the coordinator's loop counter. None of that appears in the programmer's own reasoning, which is closer to:

> The coordinator asks each participant in turn — the order does not matter — and each participant answers with its own vote. When every answer is in, the coordinator decides.

The informal argument works for the following (informal) reasons. First, the coordinator treats `send` followed by `recv` as though it were an ordinary function call: nothing observable happens in between. Second, the order in which unrelated participants are asked does not matter. Thus, correctness is based on sequential reasoning on the coordinator, with atomic and independent calls to each participant.

The methodology described here is a way to make both of those claims into checked proof obligations, so that the resulting proof looks like the informal argument rather than like a case analysis over interleavings.

2 Overview: Compositional verification

Before introducing our techniques, it is worth outlining how we verify concurrent software and what the difficulties are. We focus on safety properties, which are properties that must hold in every reachable state of the program. The basic technique to prove safety properties is to find an inductive invariant, which is a property that holds in the initial state and is preserved by every step of the program. The issue with concurrent programs is that the program's state space is huge, because it includes all possible interleavings of the threads. A "global" invariant that describes the entire program state is often too complex to find or reason about: it has to maintain the global state of all threads and the shared resources and this is usually complex.

Instead, we look for *compositional* approaches. A compositional approach allows us to reason about individual threads in isolation, and then combine those proofs to conclude properties about the whole program.

Owicki–Gries Reasoning

The classical way to verify a concurrent program compositionally is to annotate each thread with an assertion at every program point, as if it ran alone, and add additional checks that ensure that the assertions remain valid under the behavior of every other concurrently running thread. Then, we verify two different proof obligations:

- **Sequential correctness.** Each thread's annotation is a valid Hoare-style proof of that thread in isolation.
- **Non-interference.** Every assertion in every thread remains true under every atomic step of every other thread.

The second family typically leads to many checks. It has one obligation per (assertion, foreign atomic step) pair, so it grows with the product of the program sizes, and the assertions have to be phrased finely enough to be preserved by whatever another thread might do next. For a protocol, this means reasoning about the network at the granularity of individual sends and receives.

One can simplify the form of the assertions by using a *yield invariant*, which is a single predicate that every thread may assume at an interference point and must preserve at each of its own steps.

Ingredient 1: yield invariants

A *yield invariant* is a predicate over the shared state. Each thread may assume it as a safe approximation of all concurrent activity in the program on arrival at an interference point and must establish that its own actions preserve the yield invariant. The obligation is:

> **NON-INTERFERENCE**
>
> Every atomic action of the program preserves the yield invariant.
>
> ---
>
> One obligation per action of the protocol, rather than one per (assertion, foreign step) pair.

The price is that the invariant must be strong enough to carry everything a thread needs across an interference point, and weak enough that every action preserves it. Choosing it well is the main creative act in these proofs; much of Section 3 is about picking a representation of the network that makes such invariants easy to state and cheap to preserve.

In principle, we should put yield invariants between any two atomic operations of each thread since other threads may interleave at those points and change the shared state. But we can often reduce the number of non-interference checks using a combination of two additional ingredients: reduction and refinement layers.

The idea of reduction is that some interleavings are redundant: they can be rearranged into an equivalent execution that "moves" a sequence of actions of one thread next to each other and treats them as a single atomic action. In this case, we do not need to put a yield invariant between those actions, since they are treated as a single atomic action without losing any generality.

Ingredient 2: reduction, via movers

Lipton's insight is that the organization of a sequence of actions into a coarser atomic block can be decided per action. An action is a *right mover* if, whenever it is immediately followed by an action of another thread, the two can be swapped without changing the outcome; and a *left mover* if it can be swapped with an immediately preceding action of another thread.

In our setting, each thread works on its own local state and communicates by sending or receiving messages. Clearly, local computation in one thread does not interfere with other threads, so we can treat a sequence of local computation as a single atomic action. Moreover, sending a message does not interfere with other threads; in particular, sending enables other threads to receive that message, but it does not disable any other action. Conversely, receiving a message does not interfere with other threads; it disables some actions, but it does not enable any other action. In Lipton's terminology, sending is a left mover and receiving is a right mover. Thus, a handler that receives a message, performs some local computation, and sends a reply has the shape `recv` · `compute` · `send` and can be treated as a single atomic action.

The reduction rule states the following:

> **REDUCE**
>
> If a thread executes a sequence of actions consisting of zero or more right movers, then at most one arbitrary action, then zero or more left movers, that whole sequence may be treated as a *single atomic action*.
>
> ---
>
> Nothing in the executable code changes. What changes is where the proof must consider interference: only between such blocks, not inside them.

Concretely, a thread that runs `recv` · `compute` · `send` could be intercepted before or after each action. But reduction allows us to treat the whole sequence as a single atomic action, so we only have to consider interference before the receive and after the send. This corresponds to our initial intuition that each participant is an atomic function call from the coordinator's point of view and we do not have to interleave between the receive and the send.

Ingredient 3: refinement layers

The third ingredient attacks detail rather than interleaving. A *layer* is a complete program with its own set of atomic actions. A concrete layer *refines* an abstract one when every step of the concrete program is matched by a step of the abstract program:

> **REFINE**
>
> If the concrete layer refines the abstract layer, then any safety property proved of the abstract layer holds of the concrete one.
>
> ---
>
> The abstract layer's actions are typically coarser and nondeterministic: they say *that* something happened without saying exactly what. Callers reason with the abstraction; the refinement obligation is discharged once.

Layers compose, so a realistic implementation can be connected to an obviously correct specification through a chain of small steps rather than one large one.

The mechanism: ownership and tokenized state machines

The three ingredients above are classical. What decides how expensive they are is a representation choice: how the shared state is written down, and who is entitled to change it.

The choice made here is to divide the state rather than share it. A *tokenized state machine* is a declaration of a transition system whose state is split into named pieces. For each piece the Verus library generates a *token* type: a proof-mode value subject to Rust's ownership rules. A thread may read or change a piece only by presenting the corresponding token, and the declaration's invariants are proved inductive over its transitions once and for all.

Two kinds of piece matter here.

- An **exclusive** piece has one token, which cannot be duplicated. A thread holding it knows that no other thread can change that piece — not because an invariant says so, but because the permission to change it is the token, and the thread has it.
- A **persistent** piece has duplicable tokens that are never retracted. A fact recorded there is public and permanent, so it can be handed to another thread and will not become stale.

This is what makes the three ingredients cheap in what follows. Non-interference for an exclusively owned piece needs no argument, because interference on it is impossible. The mover conditions of Section 4 become properties of the *shape* of an operation rather than claims about a particular protocol. And the invariant a thread must carry across an interference point shrinks to what is genuinely shared, which here is the record of what has been sent.

The price is that the state has to be divisible in the first place: the design of Section 3 is mostly the work of choosing a division under which the facts a protocol needs are facts about pieces someone owns.

The code from here on is Rust with Verus annotations. Appendix A collects the notation, including the syntax of a tokenized state machine and a small self-contained example; a reader who knows Rust but not Verus may want to read it first.

How they fit together here

Reduction makes the atomic blocks in a protocol coarse enough that a thread's code between two interference points looks like straight-line sequential code. Yield invariants discharge what is left. Refinement layers let a component — in our case, a whole remote procedure call — be replaced by a single action, so its caller never sees the messages at all. Applied to two-phase commit, the result is that the coordinator is verified as an ordinary sequential program with an ordinary loop invariant, which is where we will arrive in Section 9.

3 Modeling the network

As a program executes, messages are sent and received. The "shared state" of a message-passing program is the state of the network, and a proof of correctness has to reason about it. How should it be represented?

The first question is not what to store but *who owns it.* A single global object holding the whole network is the obvious choice and the expensive one: every invariant mentioning it must be re-established after every step of every thread, and every fact a thread wants to carry across an interference point must be re-derived from an explicit account of what other threads might have

done. That account is exactly the non-interference obligation of Section 2, and it reappears at every program point.

Instead we divide the network into pieces and give each piece an owner. The representation has three parts, and each is chosen so that a fact about it is stable for a structural reason rather than a proved one.

SRC/TOK.RS — THE THREE PARTS OF THE NETWORK STATE

```
#[sharding(map)]            pub sent:  Map<ChanId, Seq<M>>,
#[sharding(map)]            pub recvd: Map<ChanId, Seq<M>>,
#[sharding(persistent_set)] pub was_sent: Set<(ChanId, nat, M)>,
```

The send history of a channel is owned by its sender. Under per-link FIFO a channel is identified by a source and a destination, so it has exactly one sender. `sharding(map)` gives one token per channel, and a token cannot be duplicated. A thread holding channel c's `sent` token therefore knows that nobody else appends to c — not because an invariant says so, but because the permission to append is the token, and the thread has it.

The consumption record of a channel is owned by its receiver. Same reasoning, resting on `Receiver` not being `Clone`.

What was sent is public and permanent. `was_sent` records "index i of channel c carries message m". It is *persistent*: its tokens are duplicable and are never retracted. This is how a receiver learns anything at all about a history it does not own. It cannot read the sender's `sent` token, and it does not need to: a receive hands back a witness, and a witness never goes stale.

The consequence is worth stating plainly, because it removes something that would otherwise appear on every page. *There is no environment step.* A thread does not describe what other threads may have done and re-derive its facts from that description. Facts about channels it owns cannot be disturbed; facts about other channels are persistent and cannot be retracted; and the protocol's invariant is global and inductive. Nothing has to be re-established after an interference point, which is why the code that follows an interference point in Section 8 is empty.

Naming channels

Protocols need to know that distinct participants have distinct channels, and that a request channel is never a reply channel. If a channel identifier is an opaque number these are assumptions. We make them theorems instead by giving identifiers structure.

SRC/TOK.RS — CHANNEL NAMES

```
pub struct ChanId { pub fam: nat, pub ix: Seq<int> }

pub open spec fn chan(fam: nat, ix: Seq<int>) -> ChanId { ChanId { fam, ix } }
```

A name is a family tag together with indices within that family: requests are family 0, replies family 1, and the indices say which participant and which call. Two names are equal exactly when their tags and indices are, which Verus knows without being told. A protocol writes `req_chan(j) = chan(0, seq![j])` and `rsp_chan(j,k) = chan(1, seq![j, k])`, and the distinctness facts it needs follow by two small injectivity lemmas proved once in the library.

4 Movers, and why they need no annotation

This section comes before the first protocol because of what it does *not* produce: a protocol author writes no mover annotations, declares no footprints, and proves no commutativity obligations. That is worth explaining before the reader meets a protocol declaration and wonders what is missing from it.

Reduction rests on Lipton's mover types. An action is a *right mover* if it commutes with any action of another thread that follows it, and a *left mover* if it commutes with any action of another thread that precedes it. A sequence of actions executed by one thread that has the shape

`right movers` · `one non-mover` · `left movers`

can be treated as a single atomic action: any interleaved execution can be permuted into one in which the whole sequence runs uninterrupted.

For channels the assignment is forced, and it is the opposite of what one might first guess.

> **SEND-L**
>
> **`send` is a left mover.** Appending to a channel only ever *enables* other actions, never disables one, so `x; send` can always be rewritten `send; x`. The converse fails: `send(m); recv(m)` cannot become `recv(m); send(m)`, because the message is not there yet.

> **RECV-R**
>
> **`recv` is a right mover, and it blocks.** A blocking operation violates the non-blocking side condition on left movers and can never be a left mover. It is therefore the operation at which a thread yields.

The consequence is the shape of a reduced atomic block:

`recv` · `local compute` · `send` · `send`

which is exactly the shape of an event handler. A handler loop needs one interference point per iteration, at the receive, rather than one per channel operation. Every protocol in Sections 6 to 11 has this shape, and none of them says so anywhere in its text.

Why the mover types are automatic

In a design where the network is one shared object, SEND-L is a claim about a particular protocol, and it has to be proved for each one: that an action reads and writes only within a declared footprint, that its gate is preserved by unrelated appends, and that any two actions with disjoint footprints commute. That is an obligation per action and, for commutativity, per pair of actions.

Under the division of state described in Section 2 the situation changes, because the two things a send does are both invisible to other threads for reasons that have nothing to do with the protocol. A send does exactly two things:

- it changes a token that no other thread can hold, so no other thread's step can observe the change or be affected by it; and
- it adds an element to a persistent set. Set union commutes with everything, and a persistent element is never removed, so this can only enable another action, never disable one.

Both bullets are exactly the two halves of SEND-L: appending cannot disable another action, and it cannot be observed by one. Neither mentions the protocol, the message, or the channel. The mover type is therefore fixed by the *shape* of the operation — which token it consumes and which field it extends — and is settled once in the library rather than declared per protocol. The same holds for RECV-R, whose right-moverness comes from the receive token being exclusively owned.

That is what *automatic* means here, and it is worth being precise about its limits: the reasoning above is a proof about the library's operations, not a theorem Verus checks about all executions. What Verus checks is that a protocol uses only those operations, and it enforces this by construction, since a thread cannot touch a channel whose token it does not hold. What remains assumed is Lipton's theorem, below.

This also gives a criterion for gates that can be checked by reading one declaration. A gate preserves left-moverness when it mentions only protocol constants, persistent facts, or state the thread exclusively owns. A gate that mentions another thread's mutable state does not — an action gated on *channel c is empty* would be disabled by an unrelated append, and then `send` would not be a left mover at all. Such a gate cannot be written here: a gate is given the channel it is about and that channel's own history, and has no way to name another. What an earlier design had to detect, this one cannot express.

What is still assumed — and what is not needed

Lipton's theorem itself. That pairwise commutativity licenses the reordering is a statement about all executions of the program, and there is no formal execution semantics inside the Rust program to state it against. The two bullets above are its side conditions, and they are discharged; the theorem that turns them into atomicity is not, and Section 16 says so.

It is worth being blunt about how much weight that assumption is currently carrying, which is: none. **No proof in this development appeals to atomicity.** There is no obligation anywhere that depends on a block being treated as one step. `interference_point` is empty, the mover types appear in commentary rather than in any contract, and no activity's specification states $\boxed{\text{R}}^* \cdot \boxed{\text{N}} \cdot \boxed{\text{L}}^*$.

The proofs are sound without reduction, for the reason Section 3 gives: every fact a service uses is either owned or monotone, so nothing it knows can be falsified while it waits. Reduction would be needed to say something this development does not yet say — that a particular function body performs one step of a declared abstract action — and that is the open gap of Section 16.

So the mover vocabulary here should be read as an explanation of *why the ownership discipline is the right one*, not as machinery that is running. The discipline is what is doing the work, and it can be stated on its own: a service may rely on a fact across an interference point exactly when that fact is about state it exclusively owns, or about state that only grows. Section 5 states it properly, says what the proofs do rest on, and gives the side condition that keeps it true as the model grows.

5 What the proofs actually rest on

The previous section is the historical route into this design, and it is how the technique is usually described. It is not what carries the proofs. This section says what does, because the difference matters for anyone extending the system.

The foundation is ownership, not reduction

The guarantee that makes per-thread sequential reasoning valid under interleaving comes from the *tokenized state machine* meta-theory — in substance a concurrent separation logic. If a thread can change state only by performing a declared transition with tokens it holds, and every `#[invariant]` is inductive over those transitions, then the invariants hold in every reachable state of every interleaving. That is the theorem doing the work, and it is why there are no non-interference obligations anywhere in this development.

Lipton's theorem is orthogonal to it. It would license treating a *block* as one step, and nothing proved here needs a block to be one step. The places one might expect a hidden dependence do not have one: `recv_abs`'s postcondition follows from the machine's agreement invariant rather than from atomicity; the two-phase commit's loop invariant is about an immutable uninterpreted function; the storage node performs a receive and two sends while holding every relevant token, so no intermediate state is observable by anyone.

So the mover vocabulary in Section 4 should be read as an explanation of *why the ownership discipline is the right one*, not as machinery that is running.

The principle, in one sentence

> **STABILITY**
>
> **A thread's assertions mention only state it owns exclusively, or state that only grows.** Both are stable under any interleaving, which is why sequential reasoning per thread needs no interference checks.
>
> ---
>
> Structural: a service's `wf` is a specification function over its own fields, and those fields are owned tokens and local values.

Two things make this more than an aspiration.

It is **enforced by construction rather than checked**. A service's invariant is a specification function over its own fields, so it cannot name state the service does not own. An unstable invariant is not something one has to remember to avoid; it is not expressible.

And the **escape hatch is deliberately narrow**. When a thread does need a fact about state it does not own — Section 12 is the case — the only route is a `property!` that projects the fact to a *pure predicate over values the reader can name.* A pure fact cannot later become false, so it is stable for a third reason. The `birds_eye` binding going out of scope is what prevents keeping anything else, and it is why `cause_gives` and `extra_gives2` have the shape they do.

The side condition on every future field

The principle is not automatic. It holds because of the sharding strategy chosen for each field of the machine, and every current field falls in one of four stable kinds:

`map`	one exclusive token per key
`variable`	one exclusive token
`persistent_set`	monotone; tokens duplicable, never retracted
`constant`	fixed at initialisation

> FIELD DISCIPLINE
>
> **Every field of the machine must be exclusively owned or monotone.** Adding a field with shared or fractional ownership — one another thread can change underneath a holder — makes a service's assertions unstable, and per-thread sequential reasoning unsound.
>
> Nothing checks this. It is a design side condition, and adding a field is the moment to apply it.

This is the condition to check when protocol state moves into the machine, which is the main planned extension. As designed, it holds: participant state is `map`-sharded and keyed by participant, so each owns its own, and exported facts are `persistent_set`. What would break it is a counter others can decrement, or a shared `variable`.

Where reduction would still be wanted

Not for soundness. For *abstraction*: stating that a particular function body performs one step of a declared abstract action, which is the open gap of Section 16. "One action" is an atomicity claim, and coarsening granularity between layers is what reduction buys.

There is an alternative worth naming, because this development is in a separation-logic shape wearing Civl's vocabulary, and the separation-logic technique may fit better. Instead of arguing a block is atomic, designate one operation as the *linearization point* at which the abstract state changes, and show the surrounding operations are stutters. That needs no reduction theorem, and showing an operation does not touch the abstract state is precisely what an ownership discipline is good at. Which of the two to build is an open design question rather than a settled one.

Four ideas that carry more weight than their length suggests

Monotone witnesses are the bridge. `was_sent` is the only way a thread learns about state it does not own. Provenance (Section 12) and the pairwise guarantee of Section 7 are two uses of one idea, not two mechanisms.

A gate is not a blocking condition. A send whose gate fails is a bug; an operation with no available step merely waits. Keeping them apart is what makes refinement stateable, and it is why the delivery discipline is a separate concept from the gate rather than part of it.

Structural beats proved. A single sender per channel, a single consumer, a channel opened at most once, a handle that cannot be relabelled — each is enforced by Rust's type system rather than by the solver. That costs no assumption, no invariant and no solver time, and it cannot be defeated by a weak specification. One of those was a real hole, closed this way.

Purity is the export mechanism. Anything learned about unowned state must be projected to a pure predicate or it is lost when the binding goes out of scope. This is the constraint behind `cause_gives` and `extra_gives2`, and it is the one rule in this development most easily discovered the hard way.

6 A first protocol

The smallest protocol that uses every part of the method. Two parties and two channels: *A* sends `Ping` on the request channel, *B* answers with `Pong(v)` on the answer channel, and *A* receives the answer and knows the payload is well formed. The property we want is that last clause, and the thing to watch for is that *A*'s proof of it never mentions *B*. The code is `src/examples/pingpong.rs`.

SRC/EXAMPLES/PINGPONG.RS — MESSAGES AND CHANNELS

```
pub uninterp spec fn ok(v: u64) -> bool;      // what a good payload is

pub open spec fn ping_chan() -> ChanId { chan(0, seq![]) }
pub open spec fn pong_chan() -> ChanId { chan(1, seq![]) }

pub enum Msg { Ping, Pong(u64) }
```

The channels are built with `chan`, so `ping_chan() != pong_chan()` follows from how they are named rather than being assumed; `lemma_chan_distinct` discharges it.

The property, stated once

Everything the protocol guarantees is one predicate on a message and the channel it was sent on.

```
pub open spec fn pong_ok(c: ChanId, m: Msg) -> bool {
    c == pong_chan() ==> (m is Pong && ok(m->Pong_0))
}
```

An answer is a well-formed `Pong`; nothing is claimed about the request channel. This predicate will appear twice below, in two roles, and noticing that they are the same predicate is most of understanding the method.

The protocol

A protocol is an implementation of one trait. There is no state machine to write: Section 7 describes the one every protocol shares.

SRC/EXAMPLES/PINGPONG.RS — THE WHOLE SPECIFICATION

```
pub struct PingPong;

impl NetInv<Msg> for PingPong {
    open spec fn gate(c: ChanId, s: Seq<Msg>, m: Msg) -> bool { pong_ok(c, m) }

    open spec fn wit_inv(c: ChanId, m: Msg) -> bool { pong_ok(c, m) }

    open spec fn deliverable_at(v: Seq<Msg>, i: nat) -> bool { fifo_deliverable(v, i) }

    open spec fn extra(sent: Map<ChanId, Seq<Msg>>) -> bool { true }

    proof fn lemma_gate_gives_inv(c: ChanId, s: Seq<Msg>, m: Msg) { }
    proof fn lemma_extra_init(chans: Set<ChanId>) { }
    proof fn lemma_extra_alloc(sent: Map<ChanId, Seq<Msg>>, c: ChanId) { }
    proof fn lemma_extra_preserved(...) { }
}
```

Four definitions and four proofs, and the proofs are empty. Taking them in turn:

`gate` is the condition under which a send must not *fail*. This is Civl's ρ, and it is distinct from blocking: a send whose gate does not hold is a bug in the program, whereas an operation with no available step merely waits. Here, putting a malformed answer on the answer channel is a bug. The obligation appears at every call site of `send`.

`wit_inv` is what may be concluded about a message that *was* sent. This is the yield invariant: a

thread arriving at an interference point may assume it of anything on any channel. Note that it is `pong_ok` again — the gate says what may be placed on a channel, and the invariant says everything there satisfies it. They are one predicate read at two moments, which is the usual case and is why `lemma_gate_gives_inv` is empty. Section 7 says what happens when they differ.

`deliverable_at` is the delivery discipline: given what has been consumed on a channel, which index may arrive next. Per-link FIFO pins it to the current length. Section 13 gives the other case.

`extra` is for a guarantee no statement about a single message can express — an ordering *between* messages, say. Most protocols leave it `true`; Section 7 returns to the one that does not.

Services own their endpoints

A protocol is written as a set of *services*. A service is a struct holding the endpoints it owns and its own local state; its activities are ordinary methods. An endpoint carries the token for its channel, which is what keeps the proof state out of the code:

SRC/PROC.RS

```
pub struct Out<M, Inv: NetInv<M>> {     // an outbound endpoint
    pub tx:   Sender<M>,
    pub tok:  Tracked<NetSM::sent<M, Inv>>,      // the send history for tx
    pub inst: Tracked<NetSM::Instance<M, Inv>>,
}
```

`In` is the dual over the consumption record, and `Inbox` is several inbound channels waited on together. Because the token travels with the handle, `send` is a method and takes no proof arguments at all.

Two interfaces sit on top. `Process` is a service that can block: it has an invariant `wf` and a `step`. `NetHandler` is a service that *cannot* block — it has an outbound half `tick` and an inbound half `handle`, and a driver owns the mailbox and performs the one blocking receive between them:

SRC/PROC.RS — THE DRIVER

```
h.tick();                                          // N . L*   outbound
let (k, m, Tracked(w)) = inbox.recv_any_wit();     // the ONE interference point
h.handle(k, m, Tracked(w));                        // N . L*   inbound
```

This is the difference that matters for Section 4. With `Process`, `wf` is an invariant at activity entry and exit only, so a body containing two blocking receives has an interference point in the middle where nothing is required of it. With `NetHandler`, $R \cdot N \cdot L^*$ is a property of the types, and `wf` is required across the only point where another thread can interleave.

The two are layered rather than rival: `Driven`, a handler bundled with its mailbox, *is* a `Process`, so a deployment is unchanged and adopting the handler shape is a per-service decision. The honest consequence is that the shape guarantee holds of the services that are handlers, not of the development as a whole, and a claim about it has to say which.

The two parties

SRC/EXAMPLES/PINGPONG.RS

```
impl NetHandler<Msg, PingPong> for Responder {
```

```
    open spec fn wf(&self) -> bool {
        &&& self.rsp.wf() && self.rsp.id() == pong_chan()
        &&& ok(self.val)
        &&& self.inbox_chans@ =~= seq![ping_chan()]
    }
    fn tick(&mut self) { }                      // purely reactive
    fn handle(&mut self, from: usize, m: Msg, Tracked(w): ...) {
        self.rsp.send(Msg::Pong(self.val));
    }
}

impl NetHandler<Msg, PingPong> for Initiator {
    open spec fn wf(&self) -> bool {
        &&& self.req.wf() && self.req.id() == ping_chan()
        &&& self.inbox_chans@ =~= seq![pong_chan()]
        &&& self.got ==> ok(self.seen)          // the guarantee, carried here
    }
    fn tick(&mut self) {                        // this service INITIATES
        if !self.waiting { self.req.send(Msg::Ping); self.waiting = true; }
    }
    fn handle(&mut self, from: usize, m: Msg, Tracked(w): ...) {
        match m {
            Msg::Pong(v) => { self.seen = v; self.got = true; self.waiting = false; }
            Msg::Ping    => { proof { assert(false); } }
        }
    }
}
```

Four things are worth pointing at.

The bodies carry no proof arguments. No instance, no tokens, no preconditions relating them. They are the code an engineer would write. The proof state is in the endpoints the service owns, and the invariant is `wf`.

The responder has one interference point, at the driver's receive. Its own body is a send: N · L. Nothing in the code says so; it is the shape of the type.

The gate obligation appears at the send. `Out::send` requires `PingPong::gate` of the channel and message, which here reduces to `ok(self.val)`, discharged by the responder's own invariant. Weaken the gate and the protocol's guarantee goes with it; the development includes that experiment.

The initiator's proof does not mention the responder. It sends, the driver yields, it receives, and it concludes `ok(self.seen)`. The step that gets it there is the guarantee attached to the received message, converted from the accompanying witness. *A* reasons only about what the invariant says of anything on that channel.

The `Ping` branch proves `false`: the invariant says the answer channel carries only `Pong`s, so receiving a `Ping` there cannot happen. That branch exists because Rust wants the match to be exhaustive, not because the protocol admits it.

A service that initiates is a state machine

The initiator has a `waiting` field, and that is not incidental. A service that starts an exchange cannot be written as answering a request: the state that would otherwise live between the send and the receive inside one activity becomes a field, and the two halves become `tick` and `handle`. That is what "a client is a state machine" means concretely, and it is the price of the shape being

checkable.

The same field carries something else. `Initiator`'s invariant includes

```
&&& self.waiting ==> self.req.hist().len() > 0
```

a fact established before the request and used after the reply — that is, across the interference point in between. It survives, and not because anything was proved about other threads. It is a fact about a send history this service owns, and nobody else can append to it. Section 9 returns to this: it is the whole reason a blocking exchange here needs no atomicity argument.

7 The network state machine

Section 6 gave the four definitions a protocol supplies and said what each means. This section is the other half: the single state machine that gives them their meaning. It is ordinary library code, in `src/tok.rs`, and it is shared by every protocol.

SRC/TOK.RS — THE STATE

```
tokenized_state_machine!{
    NetSM<M, Inv: NetInv<M>> {
        fields {
            #[sharding(map)]            pub sent:  Map<ChanId, Seq<M>>,
            #[sharding(map)]            pub recvd: Map<ChanId, Seq<M>>,
            #[sharding(persistent_set)] pub was_sent: Set<(ChanId, nat, M)>,
            #[sharding(variable)]       pub next: nat,
            #[sharding(constant)]       pub inv: PhantomData<Inv>,
        }
```

The first three fields are the representation of Section 3: `sent` and `recvd` divided per channel and exclusively owned, `was_sent` persistent. `next` is the allocator used by `mint` (Section 9). The last field carries the protocol as a type parameter and holds no information; it is the idiom `vstd`'s own `rwlock` uses for the same purpose.

Where each part of a protocol is used

SRC/TOK.RS — THE TRANSITIONS

```
transition!{
    do_send(c: ChanId, s: Seq<M>, m: M) {
        remove sent -= [c => s];
        require(Inv::gate(c, s, m));            // the protocol's gate
        add    sent += [c => s.push(m)];
        add    was_sent (union)= set { (c, s.len(), m) };
    }
}

transition!{
    do_recv(c: ChanId, r: Seq<M>, i: nat, m: M) {
        remove recvd -= [c => r];
        have   was_sent >= set { (c, i, m) };
        require(Inv::deliverable_at(r, i));     // the protocol's discipline
        add    recvd += [c => r.push(m)];
    }
}
```

The `remove` in each says a send consumes the channel's send token and a receive its consumption token, so only a channel's sender may send on it and only its single consumer may receive. The `have` in `do_recv` requires a witness for the message being taken: you can only receive what was sent, and this is where that is enforced. The two `requires` are the only places a protocol's own definitions enter.

The invariants are what a thread may assume:

SRC/TOK.RS — THE INVARIANTS

```
#[invariant]                            // a witness describes the history
pub spec fn agree(&self) -> bool { ... }

#[invariant]                            // the protocol's guarantee
pub spec fn protocol_inv(&self) -> bool {
    forall|c, i, m| self.was_sent.contains((c, i, m)) ==> Inv::wit_inv(c, m)
}

#[invariant]                            // the protocol's extra guarantee
pub spec fn protocol_extra(&self) -> bool { Inv::extra(self.sent) }

#[invariant]                            // nothing at or above the counter exists
pub spec fn unallocated(&self) -> bool { ... }
```

`protocol_inv` is stated over `was_sent`, which only ever grows, so no other thread can falsify it: another thread may add a witness but cannot remove one, and adding one preserves a claim about all existing witnesses provided the new one satisfies it too. That is exactly what the gate guarantees, and it is the whole of the proof the machine demands of a send:

```
#[inductive(do_send)]
fn do_send_inductive(pre: Self, post: Self, c: ChanId, s: Seq<M>, m: M) {
    Inv::lemma_gate_gives_inv(c, s, m);
    Inv::lemma_extra_preserved(pre.sent, c, s, m);
    ...
}
```

A protocol's obligations are therefore small and fixed. `gate` must be strong enough to establish `wit_inv` for the message it admits — immediate when they are the same predicate, which is the usual case. And `extra` must hold initially and be preserved by a send and by the creation of a channel.

Finally, how a thread uses the invariant:

SRC/TOK.RS — READING THE INVARIANT OFF A WITNESS

```
property!{
    learn(c: ChanId, i: nat, m: M) {
        have was_sent >= set { (c, i, m) };
        assert(Inv::wit_inv(c, m));
    }
}
```

A `property!` changes nothing. It says: hold a witness, and you may conclude `wit_inv` of what it names. The `assert` is proved from the invariant, once, in the library. `In::recv` is a receive followed by this, which is why a service is handed the guarantee along with the message and never

mentions a witness.

When the gate and the guarantee differ

They coincide whenever the gate constrains the message alone. They cannot coincide when a protocol's guarantee is about *pairs* of messages, because a witness names one message. `heartbeat.rs` is the case: each beat's sequence number must exceed every earlier one on the link. Its `wit_inv` is `true` and the guarantee lives in `extra`:

SRC/EXAMPLES/HEARTBEAT.RS

```
open spec fn gate(c: ChanId, s: Seq<Beat>, m: Beat) -> bool {
    forall|x: int| 0 <= x < s.len() ==> (#[trigger] s[x]).seq < m.seq
}
open spec fn wit_inv(c: ChanId, m: Beat) -> bool { true }
open spec fn extra(sent: Map<ChanId, Seq<Beat>>) -> bool {
    sent.dom().contains(link()) ==>
        forall|x: int, y: int| 0 <= x < y < sent[link()].len()
            ==> (#[trigger] sent[link()][x]).seq < (#[trigger] sent[link()][y]).seq
}
```

Note that the gate reads the history `s`, which is what `wit_inv` cannot do. The three `extra` lemmas are then not empty, and they are one of the two places in the example protocols where a protocol writes a real proof about its own invariant; the other is the journal ordering in `leaselock.rs`.

Stating a pairwise guarantee is only half of it. A service that *owns* the channel reads the ordering straight off its own send history; a service that does not own it needs a way to get at `extra` from code, and until recently there was none — heartbeat's guarantee was proved and unusable.

The way out mirrors the mechanism of Section 12: two witnesses in, a pure fact out.

SRC/TOK.RS — READING A PAIRWISE GUARANTEE

```
property!{
    learn_pair(c: ChanId, i: nat, j: nat, m1: M, m2: M) {
        have was_sent >= set { (c, i, m1) };
        have was_sent >= set { (c, j, m2) };
        require(i < j);
        birds_eye let s = pre.sent;
        assert(Inv::extra_gives2(c, m1, m2)) by { Inv::lemma_extra_gives2(...); };
    }
}
```

`extra_gives2` must mention only `c` and the two messages, for the same reason as `cause_gives` below: the histories are reached through a binding the reader cannot name, so anything said about them is projected away. Heartbeat's is `c == link() ==> m1.seq < m2.seq`, and its watcher — which owns no send history at all — now concludes that the beats it was handed increase.

Reading `sent` this way is sound even though `sent` is not monotone, because what leaves is a pure predicate about two messages, and a pure fact cannot later become false. Retaining anything about the histories themselves would not be sound, and the binding going out of scope is what prevents it.

All of these read one channel. A guarantee that relates a message on one channel to a message on another is outside what any of them can state, and needs the separate mechanism of Section 12.

What a protocol does not write

No state machine. No mover annotations, footprints or commutativity obligations — Section 4 gave the reason. No description of what other threads may do. And no ghost state of its own: `sent`, `recvd`, `was_sent` and the allocator are all the network is.

A protocol that needs ghost state beyond that — a phase counter, a record of local decisions — declares a *separate* state machine for it. The two are independent instances, so a token of one cannot affect the other and no obligation relates them, exactly as for two protocols sharing a thread (Section 14). The case this does not cover is protocol state that must be related to network state; that has to live in one machine, and is not expressible.

8 Interference points

SRC/TOK.RS

```
pub fn interference_point() { }
```

It takes no ghost state, promises nothing, and is not trusted. Marking a yield is documentation, not an obligation, for the reason given at the end of Section 3: there is nothing to re-establish. The heartbeat example makes the point concretely.

SRC/EXAMPLES/HEARTBEAT.RS

```
pub fn beat_and_wait(&mut self)
    requires old(self).inv(), old(self).seq < u64::MAX,
    ensures
        final(self).inv(),
        final(self).link.hist().last() == (Beat { seq: old(self).seq }),
{
    let s = self.seq;
    self.link.send(Beat { seq: s });
    self.seq = s + 1;

    interference_point();

    // Nothing to prove here. Nobody else could have appended.
}
```

The service still holds the link's `sent` token after the yield, so what it wrote is still the last thing there. The conclusion is available because the permission was never given away.

This is the general rule, and it is worth stating plainly because everything else rests on it. *Every fact a service relies on across an interference point is either about a token no other thread can hold, or about the record of what was sent, which only grows.* Facts of the first kind cannot be changed by anyone else; facts of the second kind cannot be retracted. That is why nothing has to be re-established at a yield — and, as Section 16 records, it is also why the development is sound without appealing to reduction at all.

Where a protocol's definitions are checked

A protocol states its gate, its guarantee and its delivery discipline once each, and the machine of Section 7 is the only thing that reads them. There is no second copy to keep consistent: `Inv::gate` appears in exactly one `require`, `Inv::deliverable_at` in one other, and `Inv::wit_inv` in one

invariant. The obligations relating them — principally that the gate establishes the guarantee — are the trait's proof members, discharged per protocol and checked by the machine's own inductive proof.

9 Remote procedure calls

A service that makes a request and waits for the answer sends, then receives. That is a left mover followed by a right mover — the wrong shape. The two cannot be one atomic block, so the caller yields in the middle of what it thinks of as a function call.

Civl's answer is to strengthen the gate of the receive until it cannot block, so that the pair becomes L·L and the call is atomic. This section gives that construction, and then says why this development does not use it.

Why the shape does not matter here

The reason a caller yielding mid-call is usually alarming is that an invariant it holds might be falsified while it waits. Here it cannot be. Every fact a service relies on is either about a token no other thread can hold, or about the record of what was sent, which only grows. The first kind nobody else can change; the second nobody can retract.

So a blocking request-and-reply is two atomic blocks rather than one, and that costs nothing. The ping-pong initiator carries the demonstration in its own invariant:

SRC/EXAMPLES/PINGPONG.RS

```
&&& self.waiting ==> self.req.hist().len() > 0
```

established before the request, in `tick`, and asserted again after the reply, in `handle` — across the interference point between them. It survives, and no argument about other threads was needed to make it survive.

Under `NetHandler` this is simply what an exchange looks like: `tick` sends, the driver's receive is the yield with `wf` checked across it, and `handle` reacts. There is no separate calling convention to learn.

> **NO-ATOMICITY**
>
> **A request-and-reply exchange needs no atomicity argument in this model.** Claiming the pair atomic in Civl's sense would be false — the caller genuinely yields — but no proof here appeals to atomicity.

The initiator's invariant in `src/examples/pingpong.rs`, established before the request and used after the reply.

Civl's construction, and why it is not used

For completeness, and because the technique is the interesting part of the original, here is the synchronised asynchronous call as this library expresses it. A receive whose gate guarantees the message is already present cannot block, so it is a left mover:

SRC/TOK.RS — THE ABSTRACTED RECEIVE

```
pub fn recv_abs<M, Inv: DetDelivery<M>>(r: &Receiver<M>, ..., Tracked(w): Tracked<&...>)
    -> (m: M)
    requires
        w.element().0 == r.id(),
        Inv::deliverable_at(old(tok).value(), w.element().1),
    ensures
        m == w.element().2,
```

The gate is a witness: the caller must hold a persistent token saying the reply was sent, at the index it is about to consume. Two things follow.

Delivery is deterministic without freshness. The witness names an index, and the agreement invariant of Section 3 says an index determines the message. Knowing the index is enough; no single-use channel and no separate lemma about singleton delivery are needed.

The requirement is visible in the type. `recv_abs` is available only for `DetDelivery`, a protocol that can prove at most one index is deliverable at a time. An unreliable link cannot, so a remote call is unavailable on such a link — which is correct, and is enforced by the trait bound rather than by review.

Where does the caller's witness come from? From performing the handler's action itself, at the call site:

SRC/TOK.RS

```
pub proof fn absorb_handler<M, Inv: NetInv<M>>(
    tracked inst: &NetSM::Instance<M, Inv>,
    tracked reply_cap: NetSM::sent<M, Inv>,       // consumed
    reply: M,
) -> (tracked r: (NetSM::sent<M, Inv>, NetSM::was_sent<M, Inv>))
{
    let tracked (a, b) = inst.do_send(reply_cap.key(), reply_cap.value(), reply, ...);
    (a.get(), b.get())
}
```

This is a `proof fn`, not an axiom: the caller genuinely performs the machine's own send transition, using the reply channel's send token, which it holds because it created the channel.

And that is precisely the problem. **Consuming the token means no handler running elsewhere can ever send that reply.** In Civl the handler does run and the proof merely reorders it; here the token surgery makes the handler's send impossible rather than reordered, so a program built this way would block on a real queue forever. `rpc`, `recv_abs` and `absorb_handler` are therefore marked proof-level in the source and excluded from the claim that the verified sources are the running program.

Recovering the distinction would need the reduction argument to be machinery rather than commentary — see Section 16 — so that the handler could hold its own reply capability and atomicity could come from the mover argument instead of from taking the token away.

Creating channels

A caller that mints a reply channel per call needs the channel to be fresh. Freshness used to be asserted by the allocator's specification, which nobody could check. It is now proved.

SRC/TOK.RS — THE ALLOCATOR'S INVARIANT

```
#[invariant]
pub spec fn unallocated(&self) -> bool {
    forall|f: nat, j: int, k: nat| k >= self.next
        ==> !self.sent.dom().contains(dyn_chan(f, j, k)) && ...
}
```

The allocator is an exclusively owned counter, and the invariant says nothing at or above it exists yet. Adding a key to a sharded map carries an obligation that the key is absent, and that obligation is discharged from this invariant. The library's `open_new_channel` is then a verified function.

What this costs, in one protocol

`src/examples/twophase.rs` is the two-phase commit written with a private reply channel per call and the handler absorbed at the call site. Its loop has no interference point, and the correctness argument is a loop invariant — the argument a programmer would give.

It also cannot run, and the reason is structural rather than incidental. Its reply channels are minted *by the coordinator* at call time, so a participant would need the send endpoint of a channel somebody else created — and in this model an endpoint cannot travel over a channel. Adding a participant service would not help.

That single missing capability accounts for three separate limitations: a runnable remote call with per-call reply channels, participants that arrive at run time, and any accept-a-connection pattern. Whether to add it is an open design question, recorded in `docs/plan.md`.

The runnable two-phase commit is `twophase_fanout.rs`, the subject of the next section: static per-participant channels, a real participant service, and no atomicity claim.

10 Fan-out two-phase commit

The coordinator above asks one participant at a time. A deployed one broadcasts and then gathers, stopping early on the first refusal. There is no remote call here, so nothing hides the interference: every collect is an interference point, and other participants reply while the coordinator is blocked.

SRC/EXAMPLES/TWOPHASE_FANOUT.RS — BROADCAST, THEN GATHER

```
while i < self.reqs.len()
    invariant self.inv(),
{ self.reqs[i].send(FMsg::Prepare); i = i + 1; }

while k < self.rsps.len() && all_yes
    invariant
        self.inv(),
        all_yes ==> forall|j: int| 0 <= j < k ==> vote(j),
{
    let m = self.rsps[k].recv();          // interference point
    match m { FMsg::Vote(b) => if !b { all_yes = false; }, _ => all_yes = false, }
    k = k + 1;
}
```

The coordinator is a service holding `Vec<Out>` for the requests and `Vec<In>` for the replies, so neither loop mentions a token or the machine. The guarantee about a received vote arrives with the message: `In::recv` returns it already converted from the witness.

The broadcast loop contains no interference point at all: every operation in it is a send, and sends are left movers, so the whole fan-out is one block. The gather loop has one per iteration.

What carries the proof is the yield invariant, declared as one line:

```
wit_inv(c, m) { forall|j: int| is_rsp(c, j) ==> m == FMsg::Vote(vote(j)) }
```

Participant j's reply channel carries nothing but participant j's vote. Where the remote-call version learned a participant's vote by executing that participant's action, this one learns it from the invariant. Both prove the same safety property by different routes — and only this one runs, for the reason given at the end of Section 9.

What this version still does not do

It gathers in a fixed order: participant 0, then 1, and so on. That is why a plain loop invariant suffices. A coordinator that accepts the first m of n replies in whatever order they arrive needs an induction over the set of outstanding replies, which is what inductive sequentialization provides and this development does not have.

11 Chang–Roberts leader election

A ring of n nodes with distinct identifiers. Node i receives on link $i-1$ and sends on link i; on receiving a value v it forwards if $v > \texttt{id}(i)$, declares itself leader if $v = \texttt{id}(i)$, and otherwise drops the message. The safety property is that only the node with the maximum identifier can declare itself leader.

The interest here is that the invariant is genuinely about the protocol rather than about the framework.

SRC/EXAMPLES/CHANG_ROBERTS.RS — THE RING INVARIANT

```
pub open spec fn msg_ok(j: int, m: Elect) -> bool {
    &&& 0 <= m.origin@ < n_nodes()
    &&& m.val as int == id(m.origin@)
    &&& j == (m.origin@ + m.hops@) % n_nodes()
    &&& forall|k: int| 0 <= k <= m.hops@
            ==> m.val as int >= id((m.origin@ + k) % n_nodes())
}
```

A message on link j carries its originator's identifier, has travelled `hops` steps from that originator, and dominates every node it has passed through. `origin` and `hops` are `Ghost`: they exist in the proof and are erased from the running program.

A node's whole step is R·N·L: one blocking receive, a comparison, and at most one send. One interference point. When the value equals the node's own identifier, the invariant says the message has travelled a full circle and dominated everything on the way, which is the safety property. The supporting arithmetic — that a full circle covers every node — is about modular arithmetic and identifiers, and is independent of the framework.

12 Facts that span channels

Everything a protocol has said so far is confined to one channel. A gate is given the history of the channel being written and the message being placed on it; `wit_inv` is given even less; `extra`

ranges over the send histories but is established one send at a time, by a thread that holds only the channel it is writing. This is what makes the reasoning local, and it is also a real limit. Some properties are about a message on one channel and a message on another, and no amount of strengthening a gate will express them.

The lease lock is the standard example. It is written in `src/examples/leaselock.rs` as three kinds of process exchanging asynchronous messages: a lock server that hands out increasing lease tokens, one per request; a storage node that accepts a write only if its token and sequence number strictly exceed every write already accepted, appending accepted writes to a journal; and concurrent writers that acquire a lease and then write. Lease expiry is modelled by nondeterminism rather than by time. A writer may be granted a lease at any moment, so an earlier writer's token can be superseded while that writer still believes it holds the lock, and its next write is refused. Nothing in the writer's code detects this, which is the situation a fencing token exists to handle.

SRC/EXAMPLES/LEASELOCK.RS — CHANNELS

```
pub open spec fn acq_req(w: int) -> ChanId { chan(0, seq![w]) } // writer  -> server
pub open spec fn acq_rsp(w: int) -> ChanId { chan(1, seq![w]) } // server  -> writer
pub open spec fn wr_req(w: int)  -> ChanId { chan(2, seq![w]) } // writer  -> storage
pub open spec fn wr_rsp(w: int)  -> ChanId { chan(3, seq![w]) } // storage -> writer
pub open spec fn journal()       -> ChanId { chan(4, seq![]) }  // storage -> the record
```

The safety property proved is write serialization — the journal is strictly ordered by (token, sequence) — and it is the top-level theorem of the corresponding Leslie model. It is deliberately *not* mutual exclusion: two writers may simultaneously believe they hold the lease, and the protocol is still correct. That part needs nothing new. The fencing check is a gate on `journal()`, the storage node is the only sender there and so owns that history, and the ordering is a fact about state it holds.

What does need something new is the claim that a write's token was issued by the server at all. The token arrives on `wr_req(w)`, the grant was sent on `acq_rsp(w)`, and a gate on the first cannot see the second. A forged token and a real one look identical.

Provenance

The mechanism is to make a send point at a message already sent elsewhere, and to require the sender to hold the witness for it. Since witnesses are obtained only by sending or receiving, a participant cannot point at a message it never saw. Four definitions carry this, all optional and all defaulting to nothing:

SRC/TOK.RS — THE PROVENANCE INTERFACE

```
// Does sending m on c require pointing at a message sent somewhere else?
open spec fn needs_cause(c: ChanId, m: M) -> bool { false }

// Are the messages in ‘causes‘ acceptable justification for m on c?
open spec fn caused_by(c: ChanId, m: M, causes: Set<(ChanId, nat, M)>) -> bool { false }

// What a thread may conclude about m from the fact that a justification exists.
open spec fn cause_gives(c: ChanId, m: M) -> bool { true }

proof fn lemma_cause_gives(c: ChanId, m: M, causes: Set<(ChanId, nat, M)>)
    requires Self::needs_cause(c, m), Self::caused_by(c, m, causes),
             forall|e: (ChanId, nat, M)| causes.contains(e) ==> Self::wit_inv(e.0, e.2),
    ensures  Self::cause_gives(c, m);
```

A message with `needs_cause` cannot go through `send`, whose precondition denies it. It goes through `send_caused`, which additionally takes a witness and checks `caused_by` against the message that witness names, or through `send_general`, which takes a whole set of them. There is one send transition, and it carries the justification:

SRC/TOK.RS — THE TRANSITION

```
transition!{
    do_send(c: ChanId, s: Seq<M>, m: M, causes: Set<(ChanId, nat, M)>) {
        remove sent -= [c => s];
        have   was_sent >= (causes);
        require(Inv::gate(c, s, m));
        require(Inv::needs_cause(c, m) ==> Inv::caused_by(c, m, causes));
        add    sent += [c => s.push(m)];
        add    was_sent (union)= set { (c, s.len(), m) };
    }
}
```

The `have` is what does the work. It is a guard on a token the sender must already own, so the justification cannot be invented. The corresponding invariant says every message that needed one has one:

SRC/TOK.RS — THE INVARIANT

```
#[invariant]
pub spec fn provenance(&self) -> bool {
    forall|c: ChanId, i: nat, m: M|
        (#[trigger] self.was_sent.contains((c, i, m))) && Inv::needs_cause(c, m)
            ==> exists|causes: Set<(ChanId, nat, M)>|
                    causes.subset_of(self.was_sent) && Inv::caused_by(c, m, causes)
}
```

This is inductive for the same reason the rest of the model is stable under interference: `was_sent` only grows. A justification found once cannot be removed by a later step of any thread, so a message established as justified stays justified.

In the lease lock the definitions read:

SRC/EXAMPLES/LEASELOCK.RS

```
open spec fn needs_cause(c: ChanId, m: Msg) -> bool {
    c.fam == 2 && c.ix.len() == 1 && m is Write
}
open spec fn caused_by(c: ChanId, m: Msg, causes: Set<(ChanId, nat, Msg)>) -> bool {
    exists|j: nat| causes.contains((acq_rsp(c.ix[0]), j, Msg::Granted(m->Write_0)))
}
open spec fn cause_gives(c: ChanId, m: Msg) -> bool {
    Self::needs_cause(c, m) ==> m->Write_0 != 0
}
```

Note that `caused_by` derives the expected channel from `c` rather than quantifying over writers. A writer holding some other writer's grant cannot use it, because the channel the justification must have come from is determined by the channel the write is going out on.

Reading it back

A thread that receives a write holds a witness for that message and nothing else. It recovers the cross-channel fact through a property of the machine:

SRC/TOK.RS — READING PROVENANCE OFF A WITNESS

```
property!{
    learn_cause(c: ChanId, i: nat, m: M) {
        have was_sent >= set { (c, i, m) };
        require(Inv::needs_cause(c, m));
        birds_eye let ws = pre.was_sent;
        assert(Inv::cause_gives(c, m)) by {
            let causes = choose|causes: Set<(ChanId, nat, M)>|
                causes.subset_of(ws) && Inv::caused_by(c, m, causes);
            Inv::lemma_cause_gives(c, m, causes);   // (details elided)
        };
    }
}
```

Two things here deserve comment.

The `birds_eye` binding reads the whole record, which no thread owns. That is sound only because `was_sent` is monotone, for exactly the reason given above; the same binding on `sent` would be unsound, since another thread can extend a history at any time.

The second is why `cause_gives` exists at all, and it is the part that is easy to get wrong. The justifying message is reached by an existential, and the record it is quantified over is not something the caller can name. If the property concluded `exists causes. causes ⊆ ws && caused_by(...)` directly, then what the caller receives is that statement with the record projected out, which for any concrete `caused_by` is trivially satisfiable — one need only choose a `d` and `m2` of the right shape. The whole chain would verify and establish nothing. `cause_gives` is the part that survives the projection: a predicate in `c` and `m` alone, whose proof was allowed to use the justification. Its content comes from `wit_inv(d, m2)`, available because everything ever sent satisfies the protocol's guarantee.

In the lease lock, the server's gate guarantees that an issued token is nonzero, so `cause_gives` says a write's token is nonzero — a fact about the write that the writer cannot manufacture. The storage node learns it on receipt, and it feeds the gate on `journal()`, so the end-to-end statement is that every write in the journal carries a token the server actually issued.

Four checks confirm the mechanism is not decorative. A writer that sends `Write(999, ...)` after being granted a different token fails the precondition of `send_caused`; a writer that reaches for plain `send` fails its `!needs_cause` precondition; a storage node that omits the `learn_cause` call can no longer discharge the journal's gate; and deleting the call from an otherwise correct proof makes it fail, which is what rules out the vacuous version. The first is kept as `counterexamples/forged_lease.rs`.

What this does not reach

Provenance relates a message to *some* earlier message. It does not by itself give an ordering between messages sent to different participants, so global monotonicity of the server's grants — that tokens handed to different writers increase — is still not expressible this way. Write serialization does not need it, since it follows from the fencing gate alone, and the storage node proves that without reference to the server or to any other writer.

13 Unreliable links and unordered delivery

Two variations that look similar and are not. It is worth separating them, because conflating them costs either soundness or expressiveness.

- **Who may send** determines whether a send history can be exclusively owned, and therefore whether `send` is a left mover.
- **The delivery discipline** determines what a receiver may be handed. It is a property of the receive side alone.

An unreliable link

Loss and duplication are of the second kind. A lossy link has one sender and one receiver — it is a datagram link, not a shared mailbox — so ownership is untouched and the entire difference from per-link FIFO is one line:

```
deliverable(v, i) { true }        // any index, any number of times
```

Loss needs no modelling at all: nothing in this development ever forces a delivery, so a message that never arrives leaves its receiver blocked. Duplication is the dropped index constraint, and the persistent witness is what makes re-delivery expressible — a witness is duplicable and never retracted, so handing the same one back twice is what a duplicating network does.

What survives unchanged is any invariant of the form *everything ever sent satisfies* P, together with the step from a message in a receiver's hand back to the send history. Both are statements about `sent`, which duplication and loss do not touch. `receive_one` returns `ok(p.v)` on an unreliable link with no additional work.

What does not survive is counting, and the development demonstrates the difference rather than asserting it. The same claim — that two receives came from two distinct sends — is proved for a FIFO link and rejected for a lossy one:

CLAIM	WHERE	OUTCOME
two receives come from two distinct sends	`heartbeat.rs`, per-link FIFO	verifies
the same claim, verbatim	`counterexamples/lossy_counting.rs`	rejected

A protocol that needs the count must re-establish it, by putting sequence numbers in its messages and discarding what it has already seen. The obligation appears exactly where the argument would otherwise be unsound.

Many producers, one consumer

Unordered delivery from several senders is of the first kind, and it constrains the model rather than the delivery discipline. A single send history with several concurrent appenders cannot be exclusively owned, and without exclusive ownership `send` is not a left mover.

So a bag is modelled as a *family* of channels, one per producer. Each producer exclusively owns its own history, ownership and left-moverness are restored, and per-producer order is retained — which a real multi-producer channel also retains. "Any order" moves to the receive side, where it belongs:

SRC/TOK.RS

```
pub fn recv_any<M, Inv: NetInv<M>>(
    rxs: &Vec<Receiver<M>>, ...,
    Tracked(map): Tracked<&mut MapToken<ChanId, Seq<M>, NetSM::recvd<M, Inv>>>,
) -> (res: (usize, M, Tracked<NetSM::was_sent<M, Inv>>))
```

It blocks on a family of channels and reports which one produced a message. The consumer in `src/examples/collector.rs` takes whatever arrives first, from whichever producer, and learns from the yield invariant that it is well formed.

14 Several threads in one process

A thread in this development is not a process; it is a unit of sequential control that owns endpoints. A coordinator that spawns one thread per participant is ordinary, and needs nothing new.

Sibling threads of one process are, as far as the proof is concerned, unrelated threads: each preserves its own invariant, each tolerates the others' interference, and none mentions the others. What has to be arranged is not sharing ownership but *dividing* it — and once a service is the unit of ownership, there is nothing left to divide by hand.

SRC/EXAMPLES/MULTITHREAD.RS — FORKING AND JOINING

```
pub fn run_participants(p0: Participant, p1: Participant)
    requires p0.wf(), p1.wf(),
{
    let h0 = vstd::thread::spawn(move || -> (r: Participant)
        ensures r.wf() { let mut p = p0; p.step(); p });
    let h1 = vstd::thread::spawn(move || -> (r: Participant)
        ensures r.wf() { let mut p = p1; p.step(); p });

    match (h0.join(), h1.join()) {
        (Ok(_a), Ok(_b)) => { }
        _ => { abort_on_panicked_child() }
    }
}
```

A service holds its endpoints and their tokens as ordinary `tracked` fields, so it moves into a spawned closure like any other value, and two services cannot name the same channel because there is only one token for it. An earlier version of this file carved four tokens out of two maps by hand and reassembled them on the way out; the proof was the same, and the code was not something anyone would write.

The one thing a spawn does need is an explicit `requires` on the closure. Verus otherwise infers `true` and asks the body to verify unconditionally. Stating it over the values moved in puts the obligation at the spawn site, which is where the wiring is decided.

Two limitations are worth naming. Sibling threads cannot share a receive endpoint, which is the single-consumer discipline applied consistently. And threads of one process sharing mutable state *outside* the network — a counter behind a mutex — is not modelled here at all; that is ordinary shared-memory concurrency and is orthogonal to the channel reasoning.

15 Layers

A protocol proved correct against its own state machine is still stated in terms of channels and message histories. A specification is usually not: two-phase commit, seen from above, is a record of who has voted, and a leader election is an outcome. Layered refinement connects the two.

A *layer* is a gated transition system, and nothing more:

SRC/LAYER.RS

```
pub trait Spec {
    type S;   type A;   type M;
    spec fn inv(s: Self::S) -> bool;
    spec fn gate(a: Self::A, req: Self::M, s: Self::S) -> bool;
    spec fn step(a: Self::A, req: Self::M, resp: Self::M, s0: Self::S, s1: Self::S) -> bool;
}
```

It says nothing about networks, so an abstract layer may be stated over whatever state is natural for it. A refinement relates two layers:

SRC/LAYER.RS

```
pub trait Refines<Lo: Spec, Hi: Spec> {
    spec fn abs_state(s: Lo::S) -> Hi::S;
    spec fn abs_act(a: Lo::A) -> Option<Hi::A>;    // None: refines skip
    spec fn abs_msg(m: Lo::M) -> Hi::M;

    proof fn lemma_gate(a, req, s) requires Lo::inv(s), Hi::gate(...) ensures Lo::gate(...);
    proof fn lemma_step(a, req, resp, s0, s1) requires Lo::inv(s0), Lo::step(...) ensures
    ...;
}
```

`abs_state` is an Abadi–Lamport refinement mapping: the two layers may differ in state, actions and messages. `abs_act` returning `None` means the action is invisible from above, which most internal steps of a real implementation are. The gate condition is Civl's, and runs in the direction one might not expect: the *abstract* action must fail more often, so that wherever the abstraction is safe the implementation is.

Both refinement lemmas may assume the concrete invariant, because refinement is only ever appealed to at reachable states. This matters in practice. In `src/examples/layers_demo.rs` the abstraction retains only the last sequence number while the concrete gate speaks of every element of the history; bridging the two requires knowing the history is increasing, which is exactly the invariant.

A worked refinement: the lock server

`src/examples/leaselock.rs` carries the example the machinery is for. The implementation reads a real clock and tracks the outstanding lease, refusing while that lease is still live. The abstraction does neither:

SRC/EXAMPLES/LEASELOCK.RS — THE ABSTRACT SERVER

```
impl Spec for LockHi {
    type S = u64;                       // the highest lease issued; nothing else
    type A = LockAct;                   // Grant | Refuse
```

```
    open spec fn gate(a: LockAct, req: Msg, s: u64) -> bool {
        match a { LockAct::Grant => s < u64::MAX, LockAct::Refuse => true }
    }
    open spec fn step(a: LockAct, req: Msg, resp: Msg, s0: u64, s1: u64) -> bool {
        match a {
            LockAct::Grant  => resp is Granted && resp->Granted_0 > s0
                               && s1 == resp->Granted_0,
            LockAct::Refuse => resp is Denied && s1 == s0,
        }
    }
}
```

It grants *some* higher number, or refuses, for reasons of its own. Two abstractions are happening at once: the clock and the lease holder disappear entirely, and "issues exactly one more than the last" relaxes to "issues something higher".

The clock deserves a note. `clock_now` is `external_body` with *no* `ensures` — it promises nothing about the value returned. So it costs no assumption, and every proof below it holds for any clock whatever, one that jumps or runs backwards included. Giving it a postcondition would have quietly made safety depend on clock behaviour.

The activity's own obligation is stated at the *lower* layer:

```
&&& h1 == h0.push(h1.last())
&&& exists|a: LockAct| LockLo::step(a, Msg::Acquire, h1.last(),
                                    old(self).st(), final(self).st())
```

and `lift` carries it up. Keeping the code's obligation at the lower layer is what lets the layers be restated without touching any activity body.

One practical lesson: choose the concrete state so the mapping is as close to the identity as it can be. An earlier version stored "next token to issue" and mapped it to "highest issued" by subtracting one, and the off-by-one leaked into the abstract gate, which had to read `hi < MAX - 1` for no reason a reader could see. Storing "highest issued" made the mapping a projection and the two gates coincide.

Finally, what a refinement proof does *not* catch. It constrains the implementation only. Weakening the abstract specification is trivially refined and will not be caught here — the strength of the abstract layer is what downstream proofs rely on, and has to be reviewed on its own. Both directions of Civl's condition are live, though: mapping the state to a constant fails the step condition, and offering an abstract action where the implementation would fail breaks the gate condition. The development includes both experiments.

The bottom of the stack is derived, not restated

A stack has to start somewhere, and the tempting thing is to write the bottom layer by hand: restate the protocol's gate as `gate` and its transition as `step`. That proves refinement about a *copy* of the protocol, with nothing checking that the copy agrees with the protocol the tokens verify.

SRC/LAYER.RS

```
pub trait Layered<M> : NetInv<M> {
    type St;
    spec fn st_inv(s: Self::St) -> bool;
```

```
    spec fn st_send(s0: Self::St, s1: Self::St, c: ChanId, q: Seq<M>, m: M) -> bool;
    spec fn st_recv(...) -> bool;
    proof fn lemma_send_gate_st(...);   // the state gate is the machine's own require
}

impl<M, T: Layered<M>> Spec for BottomLayer<(M, T)> { ... }
```

A protocol names its machine's own state, invariant and transition relations — one line each, pointing at what the state machine library already generated — and `BottomLayer<T>` is those. There is no second definition to keep in step. Changing the protocol's gate changes the refinement obligation, which is the property a hand-written bottom layer does not have.

An expressiveness note

`abs_act` is a function of the action alone, so an action cannot be visible in some states and invisible in others. State-dependent invisibility belongs on the abstract side instead, where the abstract transition simply admits the identity: `HbTop::step` is *the count grows, or nothing changed.* `None` is for actions that are always internal.

16 What is assumed

Verus checks that the obligations above are discharged. What it does not check is the meta-theory that makes discharging them sufficient. So the boundary has to be drawn explicitly — and drawn in the right place, which is not where an earlier version of this write-up drew it.

The load-bearing assumption is the **tokenized state machine meta-theory**: that a thread which changes state only by performing declared transitions with tokens it holds, over invariants proved inductive, cannot violate those invariants under any interleaving. That is what licenses per-thread sequential reasoning here, and it is assumed rather than proved in this development because it belongs to the framework rather than to the protocols. It is also, unlike reduction, an assumption every proof below genuinely rests on.

Lipton's theorem is also listed, and it is honest to record that *nothing currently rests on it* — see Section 5. It is kept on the list because the abstraction step this development still lacks would need it, or would need the linearization-point argument that could replace it.

ASSUMED	WHY
`send_general`, `recv`, `recv_any`	The channel primitives. Their specifications are the state machine's transitions; the runtime that implements them is not verified. `send_general` carries the witnesses for whatever messages the one being sent must point at; `send` and `send_caused` are verified wrappers over it, not further assumptions.
`make_endpoints`	Creates a channel and returns its two handles. It takes the channel's send and receive tokens and gives them straight back, which looks pointless and is not: the machine holds exactly one of each, so demanding both makes this callable at most once per channel, and pairs the two ends with each other by construction. Protocol-independent, so creating channels costs no per-protocol assumption.
`abort_on_panicked_child`	Diverges, so any postcondition holds. Used when a spawned thread panics and its tokens are therefore gone.
Tokenized state machine meta-theory	That ownership of tokens plus inductive invariants gives per-thread sequential reasoning under any interleaving. The assumption every proof here rests on (Section 5). Not an `external_body` declaration — it is a property of the framework, not of this development.
Per-protocol configuration	Four small facts, one per protocol: how many participants, nodes or producers there are, and (for the ring) that node identifiers are distinct.
Lipton's theorem	That pairwise commutativity licenses treating a block as atomic. Its side conditions are discharged (Section 4); the theorem is not. Listed for completeness: no proof here appeals to it.

Nine `external_body` declarations in total. Note what is *not* on the list, and was on the corresponding list of an earlier version of this development: channel freshness, channel distinctness, the reduction rule for executing a handler at the call site, dividing and recombining ownership across threads, and the abstracted receive. Each of those is now proved.

Two things are enforced by Rust's type system rather than by the solver, and belong here because they are load-bearing for the primitives above.

A channel cannot be opened twice. `make_endpoints` consumes both of the channel's tokens, so a second call has nothing left to pass. Were it possible, each call would create a fresh queue, and a sender from one call could be paired with a receiver from another: two unrelated queues that the model believes are one channel, every proof still valid, and no message ever delivered.

A handle cannot be relabelled. `Sender` and `Receiver` are opaque, with private fields. With the fields public, verified code could take a handle whose real queue belongs to one channel and rebuild it carrying another channel's name; a send would then satisfy every precondition, record one channel's history growing, and put the bytes somewhere else. The receiver on that queue would be handed a message that `recv`'s postcondition asserts was sent on its own channel, and was not — the one boundary everything rests on, false at run time. This was a real hole in an earlier version, found by review and closed; `counterexamples/relabel_endpoint.rs` keeps it closed.

Both are worth preferring where they are available. A property the type system enforces costs no assumption, no invariant and no solver time, and it cannot be defeated by a weak specification.

The per-protocol residue is worth looking at, because it is small and it is not about channels:

```
twophase, twophase_fanout:   n_parts() >= 0
collector:                   n_producers() > 0
chang_roberts:               n_nodes() >= 1, and node identifiers are distinct
```

These are facts about a deployment. Everything a protocol used to assume about its own channel naming — that participants have distinct channels, that a request channel is never a reply channel, that a minted channel is fresh — now follows from how names are built.

Where the assumptions could be reduced further

The remaining reduction assumption is the substantial one — but, as Section 4 says, nothing currently rests on it. Discharging it matters not because a proof is waiting on it, but because it is what would let a function body be tied to a declared abstract action. Building a shallow trace semantics as specification-level data and proving the reduction theorem about it would relocate the trust from *reduction is sound*, a claim about all executions, to *these three functions implement this transition relation*, which is smaller and reviewable. That is the natural next step, along with the inductive rule needed for the quorum-gathering protocols of Section 10.

One obligation is currently discharged by review rather than by the verifier: a function body is not tied to a declared action of the protocol. In the earlier design this connection was written out by hand for one protocol; carrying it over needs the global state that the token representation deliberately hides, and is unfinished.

17 Summary

For a reader who wants the short version of what writing a protocol involves:

1. Declare the message type and the protocol's parameters.
2. Implement `NetInv`: the gate, the guarantee about a message that was sent, the delivery discipline, and — only for a guarantee no single message can express — the `extra` invariant and its three lemmas. A guarantee that relates messages on *different* channels additionally needs `needs_cause`, `caused_by`, `cause_gives` and one lemma (Section 12); all four default to saying nothing.
3. Write each participant as a service: a struct holding the `Out` and `In` endpoints it owns and its local state, with its activities as ordinary methods and its invariant as `wf`. Prefer `NetHandler` to `Process`, so that the one interference point is the driver's and the invariant is checked across it.
4. Wire the services into a deployment. A service verified under preconditions nobody discharges proves very little.
5. Implement `Layered` if the protocol is to sit at the bottom of a refinement stack.

There is no mover annotation, no footprint, no commutativity obligation and no account of what other threads may do. Those obligations have no counterpart here: the facts they used to establish are consequences of who owns what, and the mistakes they used to catch cannot be written down.

The cost, stated fairly, has moved. It used to be notation: every function that touched the network carried the state machine instance and a channel token, with preconditions relating them. Endpoints now own their tokens, so activity bodies carry no proof arguments at all, and the deployment in

`src/examples/lease_system.rs` boots three services, runs them on three threads and reports its result — from the same source Verus checks.

What remains is narrower and less pleasant. A service holding a *vector* of endpoints must state its per-endpoint invariant over the vectors rather than over itself, and re-establish the quantifier by hand after each mutation, because a fact about a receiver does not survive a call that changes it. That is per-service friction the abstraction ought to absorb and does not.

The limit of the approach is that the network is exactly `sent`, `recvd`, the record of what was sent, and the allocator. A protocol needing ghost state of its own declares a separate state machine, which composes with this one at no cost because the two are independent instances. What is not expressible is protocol state that must be *related* to network state; that would have to live in one machine. Relating messages on different channels to each other, which was outside the model in an earlier version of this development, is now within it, and at no cost in assumptions: the send primitive was generalised rather than duplicated.

Two consequences of that limit are worth naming, because they are the properties a reader is most likely to go looking for. Global monotonicity of the lease server's grants, and mutual exclusion as opposed to write serialization, are both outside it: each needs state that is the server's own, related to what the network has carried. And an endpoint cannot be sent over a channel, which is what keeps a remote call with per-call reply channels from running, and what keeps participants from arriving while the program does.

A Verus notation

The development is written in Rust; Verus adds specifications and proofs to Rust and discharges them with an SMT solver. This appendix collects what a reader who knows Rust but not Verus needs in order to read the code in the body.

VERUS NOTATION

`spec fn`	A mathematical function, usable only in specifications. Erased before compilation.
`proof fn`	A lemma. Its `requires`/`ensures` are its statement; its body is its proof. Erased before compilation.
`open spec fn`	A specification function that can be called in proofs, but not in executable code. The difference between "open spec fn" and "spec fn" is that the former can be called in proofs, while the latter can only be called in specifications.
`fn`	Executable code. This is what actually runs.
`requires/ensures`	Pre- and postconditions.
`old(x), final(x)`	For a parameter passed by mutable reference, its value before and after the call. Both appear in postconditions.
`Ghost<T>`	A value of type `T` that exists only in the proof and occupies no space at runtime. For a value `x` of type `Ghost<T>`, we write `x@` to read the underlying value.
`Tracked<T>`	Like `Ghost`, but subject to Rust's ownership rules: it cannot be duplicated or discarded silently.
`Seq, Set, Map`	Mathematical sequences, sets and maps. Unbounded; specification-only.
`&&&, |||`	Bulleted conjunction and disjunction, for readable multi-line formulas.

`x->Variant_0`	Projects the first field of an enum variant. Total: applying it to the wrong variant is well-typed and yields an unspecified value.
`uninterp`	An uninterpreted specification function: a parameter of the protocol, constrained only by whatever axioms are stated about it.
`external_body`	Marks a function whose body is *not* verified. Its specification is assumed. Every one of these is an axiom.
`tracked x`	A proof-mode value subject to ownership: it can be moved and consumed, but not copied.

One Rust feature matters to the proof rather than to the syntax. Values have unique owners, and a value that is not `Clone` cannot be duplicated. The development gives `Receiver` no `Clone` implementation, so a receive endpoint is owned by exactly one thread, and Rust's type system discharges the single-consumer condition the proof needs at no cost. Section 3 applies the same idea to the network state itself.

Tokenized state machines

The one Verus library the development relies on is `tokenized_state_machine!`. Section 2 says why this shape of declaration is useful; this appendix says what it looks like and what it generates.

The macro takes a transition system whose state is divided into named *fields*, each annotated with a *sharding strategy* saying how that field is divided among its owners. Here is a complete example: a counter that several threads may increment, where each holds a share of the total.

A SMALL TOKENIZED STATE MACHINE

```
tokenized_state_machine!{
    Counter {
        fields {
            #[sharding(variable)] pub total: nat,   // one exclusive token
            #[sharding(count)]    pub units: nat,   // splittable into shares
        }

        #[invariant]
        pub spec fn agree(&self) -> bool { self.total == self.units }

        init!{
            boot() { init total = 0; init units = 0; }
        }

        transition!{
            incr() {
                update total = pre.total + 1;       // change an exclusive field
                add    units += (1);                // mint one share
            }
        }

        #[inductive(boot)]  fn boot_inductive(post: Self) { }
        #[inductive(incr)]  fn incr_inductive(pre: Self, post: Self) { }
    }
}
```

From this the macro generates:

- a type `Counter::Instance`, identifying one running copy of the machine, together with

`Instance::boot()`, which returns the instance and *every token of the initial state*;

- a token type per field — `Counter::total`, `Counter::units` — whose values are `tracked`, so they obey Rust's ownership rules;
- a proof method per transition, here `instance.incr(...)`, which consumes the tokens the transition removes and returns those it adds;
- a proof obligation per `#[inductive(..)]` stub, discharged by the bodies above, that the transition preserves every `#[invariant]`.

The strategies used in this development are `variable` (a single exclusive token for the whole field), `map` (one exclusive token per key), `constant` (a value fixed at initialisation, readable from the instance), and `persistent_set` (duplicable tokens that are never retracted). Within a transition body:

CLAUSE	MEANING
`require(P)`	The transition may only be taken where `P` holds. Whoever performs it must establish `P`.
`remove f -= [k => v]`	Consume the token for key `k`, whose value is `v`. The performer must hold it.
`add f += [k => v]`	Produce a token for `k`. For a map, this carries an obligation that `k` was absent.
`have f >= {x}`	Read a persistent token without consuming it.
`update f = e`	Change a `variable` field.
`assert(P)`	A proof obligation discharged using the invariants.
`birds_eye let x = e`	Bind a value read from the whole state, including fields the performer does not own. The binding is not available to the caller, so only what is proved *without* mentioning it is returned (Section 12).

A `property!` block has the same shape as a transition but changes nothing. It is how a thread converts tokens it holds into a consequence of the machine's invariants — the mechanism Section 8 uses to read a yield invariant off a witness.

Two consequences are worth keeping in mind while reading the body. A thread can affect the state only by performing a declared transition, and only if it holds the tokens that transition consumes; and the invariants are established once, over the declared transitions, rather than re-proved at each use.

Sources. The proof rules are Civl's. See Kragl and Qadeer, *The Civl Verifier* (FMCAD 2021); Kragl, Qadeer and Henzinger, *Refinement for Structured Concurrent Programs* (CAV 2020); Kragl, Enea, Henzinger, Mutluergil and Qadeer, *Inductive Sequentialization of Asynchronous Programs* (PLDI 2020); and Gangamreddypalli, Enea and Qadeer, *Reduction for Structured Concurrent Programs* (ESOP 2026).

The library is `src/tok.rs` and `src/layer.rs`. The protocols are in `src/examples/`; `counterexamples/` holds protocols that must fail to verify. Checked with Verus 0.2026.08.15.